Basketball Statistics Processing Using Akka Streams

Zakaria Rabahi

December 31, 2024

Introduction

The aim of this project is to process a large dataset of NCAA basketball games using Akka Streams. The dataset contains historical records with detailed game information. By leveraging the Pipe-and-Filter architectural pattern, the system efficiently processes the data in a streaming fashion to answer key statistical questions. These include counting Sunday victories, close victories, quarter-final appearances, and losses during a specific decade. This project demonstrates the modularity and scalability of Akka Streams for real-time data processing.

General Architecture

The system follows a modular Pipe-and-Filter architecture, which separates each processing stage into distinct components. This approach allows for efficient and reusable components. The main stages of the architecture include:

1. Reading the CSV file and converting it into a stream of records.
2. Parsing the records into domain-specific objects (`GameRecord`).
3. Applying filters and aggregation logic via custom flows.
4. Combining results from parallel pipelines.
5. Writing aggregated results to external files.

This architecture ensures scalability and supports high-throughput processing by distributing work across parallel pipelines using Akka Streams' `Balance` stage.

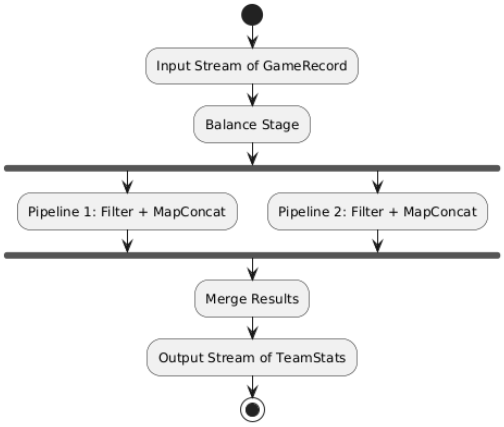

Figure 1: Overall Architecture of the System.

Component Explanation

Parser

The parser is responsible for converting raw CSV rows into `GameRecord` objects. This step involves extracting relevant columns, such as the season, game date, winning team, and losing team. By encapsulating parsing logic in a single stage, the parser ensures that the subsequent stages work with well-defined data structures. This modular design simplifies debugging and enhances maintainability.

Flows

Custom flows are at the core of the system's processing logic. Each flow applies specific filters to the stream of `GameRecord` objects. For example:

- The Sunday victories flow filters records where `dayOfWeek == "Sunday"`.
- The close victories flow filters records where `pointsDifference <= 5`.
- The quarter-finals flow filters records based on the `round <= 4` condition.
- The losses flow focuses on games from the 1980s.

These flows use the `Balance` stage to distribute the workload across two parallel pipelines, ensuring efficient processing.

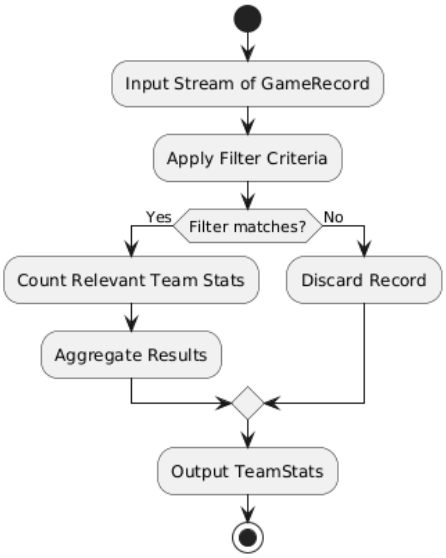

Figure 2: General UML for Filters.

Pipelines

Each pipeline within the flows performs independent processing. By filtering and counting relevant records, the pipelines produce intermediate results, which are then merged into a single stream. This parallelization not only improves performance but also makes the system resilient to uneven workloads.

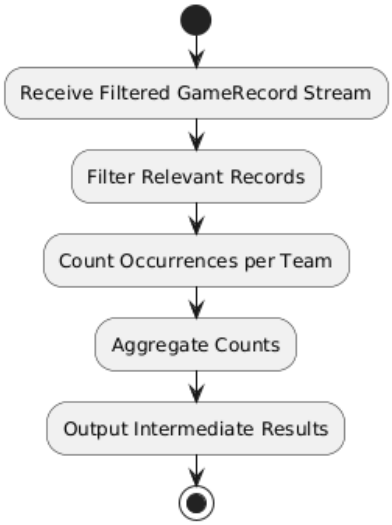

Figure 3: Detailed Pipeline Diagram.

Aggregator

The aggregator consolidates the results from the pipelines. It groups records by team and calculates the desired statistics, such as the number of wins, losses, or appearances. This aggregation ensures that the final output provides meaningful insights derived from the dataset.

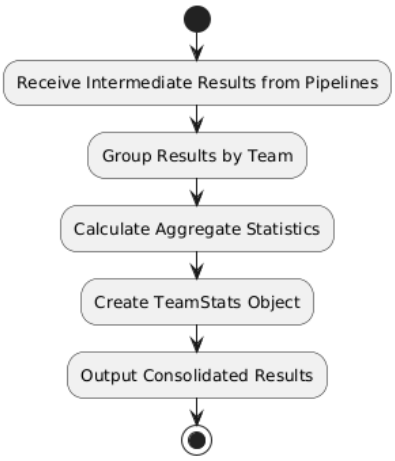

Figure 4: Aggregator Process UML.

Quarter-Finals Flow

The quarter-finals flow focuses on identifying games where teams reached at least the quarter-final stage. It employs a balanced flow structure but includes logic to count both wins and losses for each team. This ensures a comprehensive aggregation of team performance at the quarter-final level.

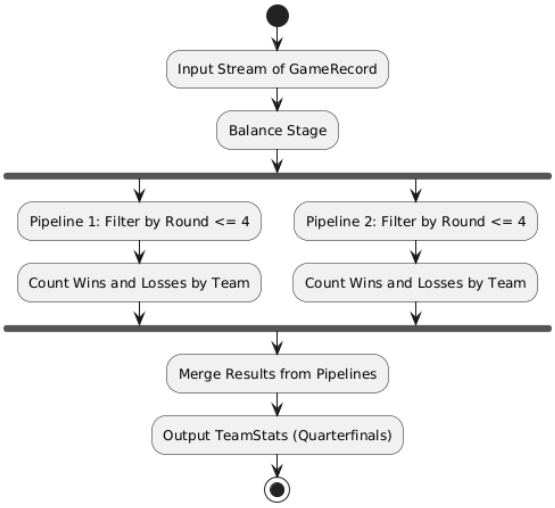

Figure 5: Quarter-Finals Flow UML.

The flow filters records with rounds less than or equal to 4, representing the quarter-finals and earlier stages. Each pipeline processes half of the filtered data, counting occurrences for both winning and losing teams. The results from the pipelines are merged into a consolidated output stream.

File Writing Process

The file-writing process ensures that aggregated results are written efficiently and accurately. After data validation and formatting, results are saved to external files.

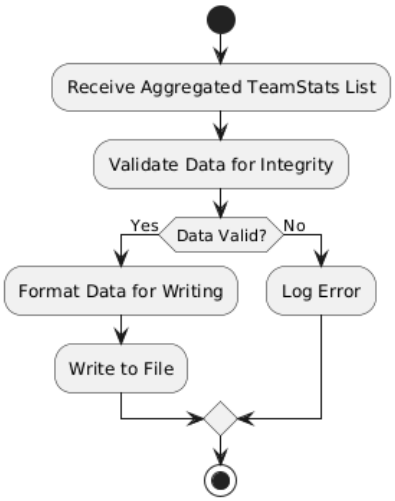

Figure 6: File-Writing Process UML.

Why Use Lists Before Writing to Files?

The system first aggregates results into lists before writing them to files instead of writing directly in the sink. This design decision was made for the following reasons:

- **Decoupling of Logic**: By separating data aggregation and file writing, the system maintains cleaner separation of concerns. The sink stage focuses solely on writing data.
- **Buffering and Optimization**: Collecting results in memory allows the system to perform bulk writes to files, which is more efficient than writing records one by one.
- **Scalability**: This approach simplifies the logic for scaling the file-writing stage. If additional post-processing is required, it can be easily integrated without modifying the sink logic.

This design choice enhances the modularity and maintainability of the system.

Conclusion

The Akka Streams-based system effectively processes large datasets using a modular and scalable Pipe-and-Filter architecture. By employing custom

flows, parallel pipelines, and efficient aggregation strategies, the solution answers specific statistical queries while ensuring high performance. The decision to aggregate results in memory before writing to files ensures optimal efficiency and clean separation of concerns. Overall, the system demonstrates the power of Akka Streams for real-time data processing in modern software architectures.

References

1. Akka Streams Documentation: `https://doc.akka.io/docs/akka/current/stream/index.html`

2. Alpakka Documentation: `https://doc.akka.io/docs/alpakka/current/index.html`